

Vector Databases & RAG

Embeddings recap · vector DB fundamentals · Qdrant hands on · RAG from scratch

1. Topic Overview

Section	What it covers
Embeddings recap	Static versus contextual embeddings, dense versus sparse representation, why transformer based embeddings won
Vector databases	What problem they solve that a traditional database cannot, the id, vector, payload record shape, similarity metrics, the storage sizing formula
Qdrant hands on	Collections and points, inserting with upsert, querying with query_points, score thresholds, payload filtering
RAG from scratch	Fine tuning versus RAG, the three RAG phases, a full ingestion and retrieval and generation pipeline

Industry Context

An estimated 30 to 40 percent of the AI projects Codebasics builds for clients are RAG projects, and it remains one of the most requested patterns across the AI industry.

2. Key Concepts

2.1 Embeddings, Quick Recap

- **Embedding.** A numeric vector representation of text, built so the numbers capture meaning.
- **Static embedding** (e.g. Word2Vec): one fixed vector per word, regardless of context. Breaks on words with more than one meaning, "bank" in "bank of the river" versus "bank" in "withdraw money from the bank" gets the same vector either way.
- **Contextual embedding** (produced by a transformer's encoder): the vector for a word changes depending on the sentence it sits in, solving the problem above.
- **Dense vs. sparse.** Modern sentence transformer models produce dense vectors (most values non-zero, compact, information-rich). One-hot encoding is sparse (most values are zero).

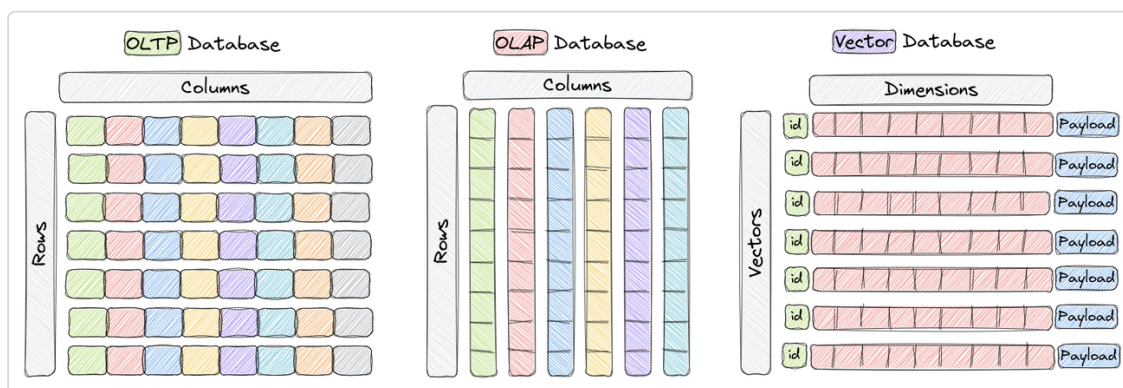
Key Insight

Two semantically similar sentences typically score 0.5 to 0.6 on cosine similarity; unrelated sentences score close to 0. This single fact is the foundation vector search and RAG are both built on.

2.2 Why a Vector Database

A traditional relational database matches by exact value or keyword. A query for "books about travelling to other planets" returns nothing if the stored text never uses the word "travelling," even though a matching book exists.

- **The fix.** Embed the searchable text and the query the same way, then compare with cosine similarity. Whichever stored vector is closest in meaning wins, no shared keyword required.
- **A vector database record has three parts:** an **id** (must be unique), the **vector** itself (its length is the **dimension**), and a **payload**, any extra retrievable data such as the original text, a category, or a source document id.
- Every point in one collection shares the same vector dimension. A collection is Qdrant's equivalent of a table.

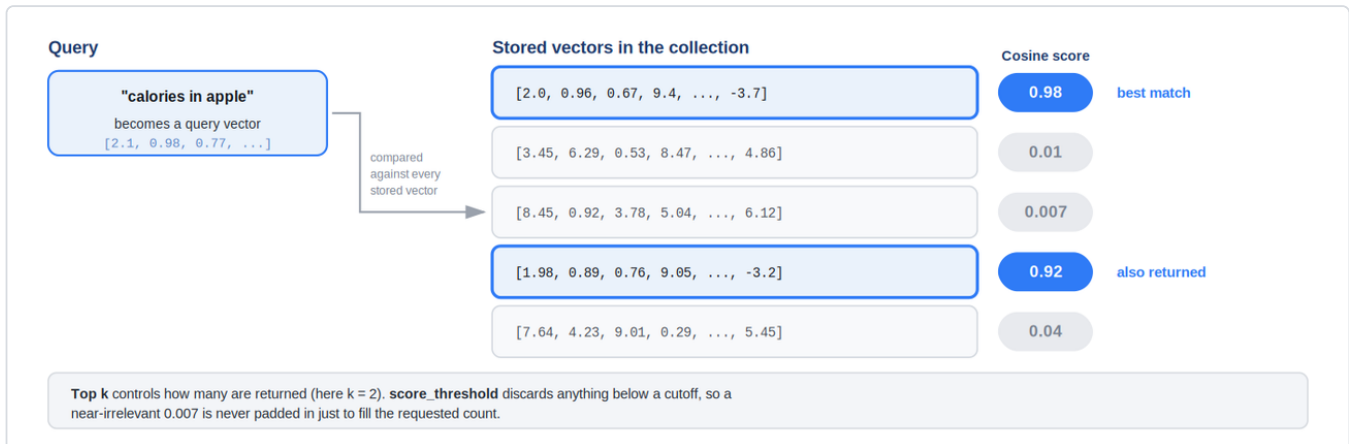


OLTP databases (MySQL, Postgres) store rows of columns. OLAP databases store the same data column by column for fast aggregation. A vector database stores id, a dense vector (the "dimensions"), and a payload per row.

2.3 Comparing Vectors: Similarity Metrics

Metric	What it measures	When to use it
Cosine similarity	Angle between two vectors: dot product divided by the product of their magnitudes	Default choice. Match whatever the embedding model was trained on; if that is unknown, cosine is the safe default
Euclidean distance	Straight line distance between two points	When absolute magnitude, not just direction, matters
Manhattan distance	Sum of absolute differences along each dimension	Less common for embeddings, more of a general distance metric

2.4 Qdrant Hands On: Collections, Points, and Filtering



Search ranks every stored vector against the query by cosine score, then returns the strongest matches.

- Qdrant has no tables or rows. Its equivalents are the **collection** (a named set of points sharing a dimension, like a table) and the **point** (one row: id, vector, payload).
- A collection is created with a vector size and a distance function, for example 384 dimensions and cosine distance for the all-MiniLM-L6-v2 model.
- **Insert:** an **upsert** call, taking a list of point structs. **Query:** **query_points**, taking a query vector and a *limit* for how many top matches to return.

Traditional database concept	Qdrant equivalent
Table	Collection
Row	Point
Insert	Upsert
Select / query	query_points
WHERE clause	query_filter, using must, should, and must_not conditions

- **Payload filtering.** Because similarity search is computationally expensive, Qdrant filters on payload fields first: **must** (AND), **should** (OR), **must_not** (NOT), the same semantics as MongoDB or Elasticsearch.
- **Example.** A filter like `category = technology` excludes non-matching points before cosine similarity even runs. The same mechanism implements role based access: a role field in the payload gates which points are searchable at all.
- **score_threshold** discards any match below a chosen similarity score. Without it, `query_points` still returns the requested count even if the best available match is nearly irrelevant, for example a score of 0.003 padded in just to fill a limit of 3.

Filter operator	Equivalent to	Effect
must	AND	Every condition listed has to match for a point to qualify
should	OR	At least one condition should match
must_not	NOT	Excludes points where the condition matches

Key Insight

Use the same embedding model for indexing and for querying. Two different models, even with the same output dimension, place vectors in different semantic spaces, so a query vector from one model will not reliably match documents embedded by another.

2.5 Sizing a Vector Store

The formula

Storage in bytes = number of records × embedding dimension × precision bytes. Then add 20 to 30 percent overhead for the database's own index structures.

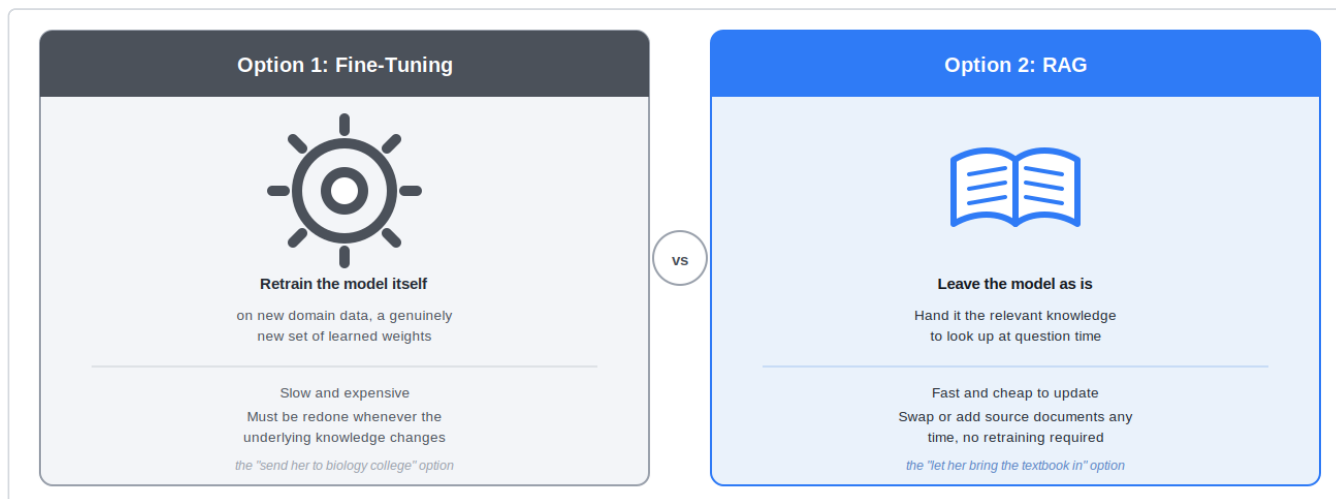
- **Precision bytes** depends on the floating point format: FP32 = 4 bytes per value, FP16 = 2 bytes.
- **Worked example.** 100,000 records × 384 dimensions × 4 bytes (FP32), then add 20 to 30 percent overhead for the vector index.
- **Payload storage is separate:** average KB of stored text or metadata per record × number of records.
- A larger embedding model (bigger dimension) directly increases both compute and storage cost, one reason the embedding model choice is not one to revisit casually mid-project.

2.6 Retrieval Augmented Generation (RAG)

- LLMs are frozen at their training cutoff and have no access to private data.
- **RAG** lets an LLM answer questions about information it was never trained on: look up relevant external knowledge at question time, then use it to generate the answer.

Fine Tuning vs. RAG

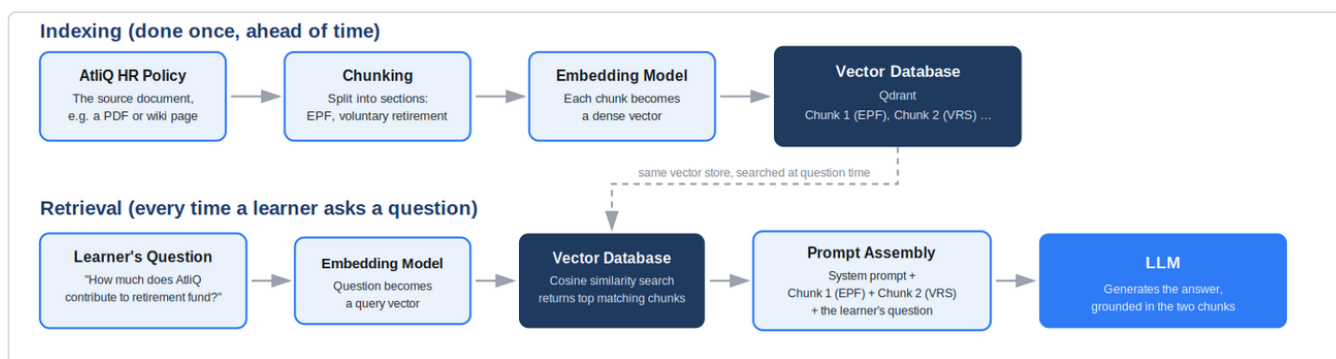
Two ways to give an LLM new knowledge. **Option 1, fine-tuning:** retrain the model itself on the new data, slow, expensive, and repeated whenever the knowledge changes. **Option 2, RAG:** leave the model as is, and hand it the relevant knowledge to look up at the moment of the question, the "open book exam" approach.



Fine tuning changes the model itself; RAG leaves it untouched and supplies the knowledge at question time.

- **Ingestion:** load the source knowledge, split it into chunks, generate an embedding vector per chunk, store embeddings plus payload in a vector database. (The storing sub-step alone is called **indexing**.)
- **Retrieval:** embed the user's question with the same model, search the vector database, pull back the top matching chunks (often 3 to 5, tunable).
- **Generation:** assemble a prompt from a system prompt, the retrieved chunks, and the question, then send it to an LLM to produce the final answer.

Worked example, AtliQ HR policy: the document is chunked into sections (EPF, voluntary retirement, and so on). For the question "how much money does AtliQ contribute to an employee's retirement fund," retrieval pulls the EPF chunk and the voluntary retirement chunk, assembles them into a prompt with the question, and the LLM answers from those two chunks alone.



Indexing runs once and builds the vector store; retrieval runs on every question against that same store.

Coding component, not covered here

A live coding walkthrough of this pipeline in plain Python was part of the session. No notebook was available to verify against, so the code is not reproduced here.

2.7 Benefits of RAG

Benefit	Why it matters
Lower cost	Passing only the relevant chunks instead of an entire knowledge base keeps the prompt, and therefore the token bill, small
Less hallucination	Answers are grounded in retrieved source text rather than the model's own possibly outdated or invented recall
Up to date knowledge	Refreshing the vector store with new documents updates what the system can answer, without retraining the LLM itself

3. Quick Revision Summary

Term	What it means
Static embedding	One fixed vector per word regardless of context, for example Word2Vec
Contextual embedding	A vector that changes based on surrounding words, produced by a transformer's encoder
Dense vs sparse	Dense: most values are non zero, compact and information rich. Sparse: most values are zero, like one hot encoding
Vector database	A database that stores vectors plus a payload and finds records by semantic similarity, not exact value
Dimension	The length of a vector, fixed by the embedding model that produced it (384 for all-MiniLM-L6-v2)
Payload	Any extra, non vector data stored alongside a point, such as original text or a category
Collection	Qdrant's named group of points that share a dimension, equivalent to a table
Point	Qdrant's unit of storage: an id, a vector, and a payload, equivalent to a row
Cosine similarity	The default metric for comparing two vectors by the angle between them
score_threshold	A minimum similarity score below which a match is discarded, even if the result limit is not yet filled
Vector store sizing	Storage in bytes = number of records × embedding dimension × precision bytes, plus 20 to 30 percent overhead
Fine tuning vs RAG	Fine tuning retrains the model on new knowledge; RAG leaves the model as is and hands it the knowledge at question time
RAG	Retrieval augmented generation: looking up external knowledge to ground an LLM's answer
Ingestion / Indexing	Ingestion is the full prep of knowledge for retrieval: load, chunk, embed, store. Indexing is the storing step within it
Chunking	Splitting a document into smaller pieces so it fits within a model's context and is easier to retrieve precisely
System prompt	The static, developer set instruction that defines an LLM's role and rules, cacheable for lower cost